Will Young
Student ID: 924230057
Week 7 Report
3/13/2026

Hello Professor,

Below is an update for my week 8 progress. In addition, I have pushed my updates to my github repo: https://github.com/Woyoung21/LastMile-Sec. For week 8, the main tasks were to populate our RAG database with the corpus of hardening documents we gathered last week as well as begin to develop the logic to inject our client environment vendors into the context for our RAG query. This was my first time creating a RAG database from scratch with Neo4j so I will spend the majority of this update talking about that development. I will start discussing the documents I chose and then dive into how I was able to populate the database.

I ended up populating the database with 14 different vendors and frameworks. These vendors form, in my opinion, the digital backbone of most small to medium enterprise organizations. Starting with cloud services and software, to server and endpoint operating systems as well as the primary web browsers, dns services, popular virtualization software, network infrastructure and the NIST regulatory framework. Overall, I populated the database with roughly 4200 security controls, and I may add more as we continue.

Vendor	Total Count
NIST SP 800-53	1196
Windows Server	671
Microsoft Edge	428
Google Chrome	386
M365	316
Windows 11	260
Ubuntu Linux	202
Google Cloud (GCP)	193
Palo Alto Firewall	180
macOS	162
VMware ESXi	77
Infoblox DNS	62
Mozilla Firefox	33
Cisco Switch	26

Will Young
Student ID: 924230057
Week 7 Report
3/13/2026

To bring these into the database, I developed a single-pass Cypher query that performs a three-way filter: mapping MITRE techniques to NIST controls, filtering by the client's specific technology stack, and calculating vector similarity using Gemini's gemini-embedding-001. As I did with Section 1 and Section 2, I am enriching our baseline JSON with new metadata fields, this way we don't lose any important information from section to section. For section 3, we append a section called rag_correlation directly to the metadata field. By doing this, we are maintaining all information from parsing the input all the way to correlation. Below is an example of the section. This is a work in progress, and I will be doing more testing and fixing as we progress. To only retain pertinent information, I sanitized irrelevant fields like the 768-dimension vector embeddings for each event because that would have blown up the size of our total file drastically.


```
rag_correlation": {
          "similarity_score": 0.8238239288330078,
          "vendor_name": "Windows Server",
          "best_control": {
            "control_id": "2.3.10.4 (L1) Ensure 'Network access: Let Everyone
permissions apply to anonymous users' is set to 'Disabled' (Automated)",
            "updated_at": "2026-03-20T14:57:06.542000000+00:00",
            "audit_procedure": "Navigate to the UI Path articulated in the
Remediation section and confirm it is set as prescribed. This group policy
setting is backed by the following registry location:
HKEY_LOCAL_MACHINE\\SYSTEM\\CurrentControlSet\\Control\\Lsa:EveryoneIncludesAno
nymous",
            "created_at": "2026-03-20T14:57:06.542000000+00:00",
            "remediation_steps": "To establish the recommended configuration
via GP, set the following UI path to Disabled: Computer
Configuration\\Policies\\Windows Settings\\Security Settings\\Local
Policies\\Security Options\\Network access: Let Everyone permissions apply to
anonymous users",
            "vendor_product": "Windows"
          }
```


For parsing the documents, I used another LLM extractor using Gemini-2.5-Flash. We load each PDF with PyMuPDF and segment it into multi-page batches of size 5. For each batch, we call Gemini 2.5-Flash through LangChain with low temperature. We then structure output into a Pydantic schema (SecurityControl: control identifier, vendor/product, remediation text, optional MITRE techniques and audit text). That turns unstructured PDF text into normalized records we can merge into Neo4j. Remediation text is then embedded with Gemini's embedding model (gemini-embedding-001, 768 dimensions) for vector search, which is separate from the extraction model. I had to process each document one by one, as the LLM would sometimes refer to the vendor in different titles, like Linux, or Linux Ubuntu, or Linux Server. Each new title would create a new node, which caused node sprawl. So I have to merge all the nodes for each

vendor before moving onto the next document/vendor. I'm fairly certain this happened because it was my first time doing this from scratch, but it made me a lot more comfortable working in the Neo4j UI.

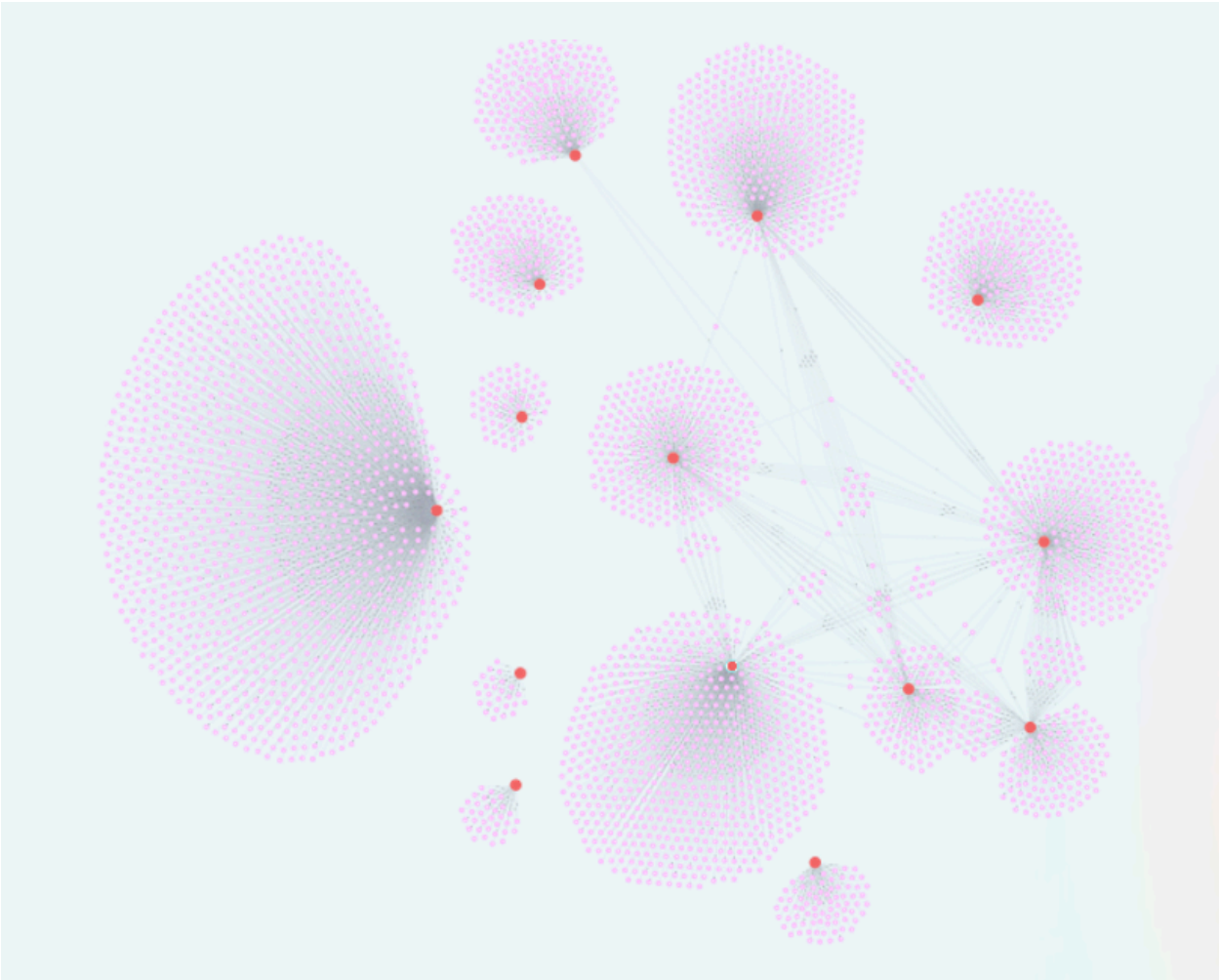

Image 1: Node visualization of graph database with node relationships.

After we finished the ingestion process, I created a visualization of all the vendor's nodes and their respective controls. The larger nodes are the vendors, and small pink nodes are the control nodes. As you can see, some nodes have correlations/relationships with each other. This is primarily because different vendor products share similar controls, which should be expected. There could be overlap between Windows 11, Windows Server, M365 as an example.

As a quick aside, there is a GLOBAL_TECH_STACK variable where we define the vendors/frameworks of the client. This is then utilized when querying the RAG database, where other vendors are ignored if they aren't explicitly mentioned.

Next week is week 9 as well as Spring Break. I did not account for that, so I will stay on track with my timeline. The goals for next week are to focus on remediation generation by building the LLM interaction that takes the section 2 output and client tech stack context to retrieve

vendor-tailored remediation steps. I did build a test file already, however it needs some work. Thanks again and I'll touch base next week.